ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп’ютерних технологій

ЗВІТ

про виконання лабораторної роботи №1

«Організація професійного Python-проєкту та Git»

з дисципліни «Професійний Python»

Виконав: студент групи ФеП-33с
Цястун Павло
Варіант 1
Перевірив:
професор Свелеба Сергій Андрійович

Львів – 2026

1. Мета роботи

Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру проєкту за принципом src-layout, працювати з модулями та пакетами Python, створювати файл pyproject.toml, використовувати Git для контролю версій та оформлювати базову документацію проєкту.

2. Номер індивідуального варіанта та постановка задачі

Варіант 1. Система обліку студентів.

Необхідно створити консольний Python-застосунок для роботи зі студентами. Для кожного студента зберігаються: ім’я, прізвище, група, середній бал. Застосунок має реалізовувати:

- додавання студента;
- виведення студентів;
- пошук за групою;
- визначення студента з найбільшим середнім балом;
- обчислення середнього бала групи.

Додатково до загальних вимог (src-layout, не менше трьох модулів, data model, окремий модуль бізнес-логіки, main.py, type hints, README.md, .gitignore, pyproject.toml, Git-репозиторій) реалізовано завдання підвищеної складності:

- введення даних із клавіатури та menu-driven interface (режим --menu);
- валідацію вхідних даних;
- сортування студентів за середнім балом;
- пошук за декількома критеріями (група та мінімальний бал);
- статистичний показник — середній бал групи;
- окрему конфігураційну константу (модуль config.py);
- власний exception InvalidStudentDataError;
- використання dataclass для моделі Student;
- README з прикладом запуску програми;
- автоматизовані unit-тести (модуль unittest).

3. Структура проєкту

Проєкт організовано за принципом src-layout: увесь програмний код розміщено в каталозі src, тести — в окремому каталозі tests.

```
student_manager/
├── pyproject.toml
├── README.md
├── .gitignore
├── src/
│   └── student_manager/
│       ├── __init__.py
│       ├── config.py
│       ├── exceptions.py
│       ├── models.py
│       ├── services.py
│       └── main.py
└── tests/
    └── test_services.py
```

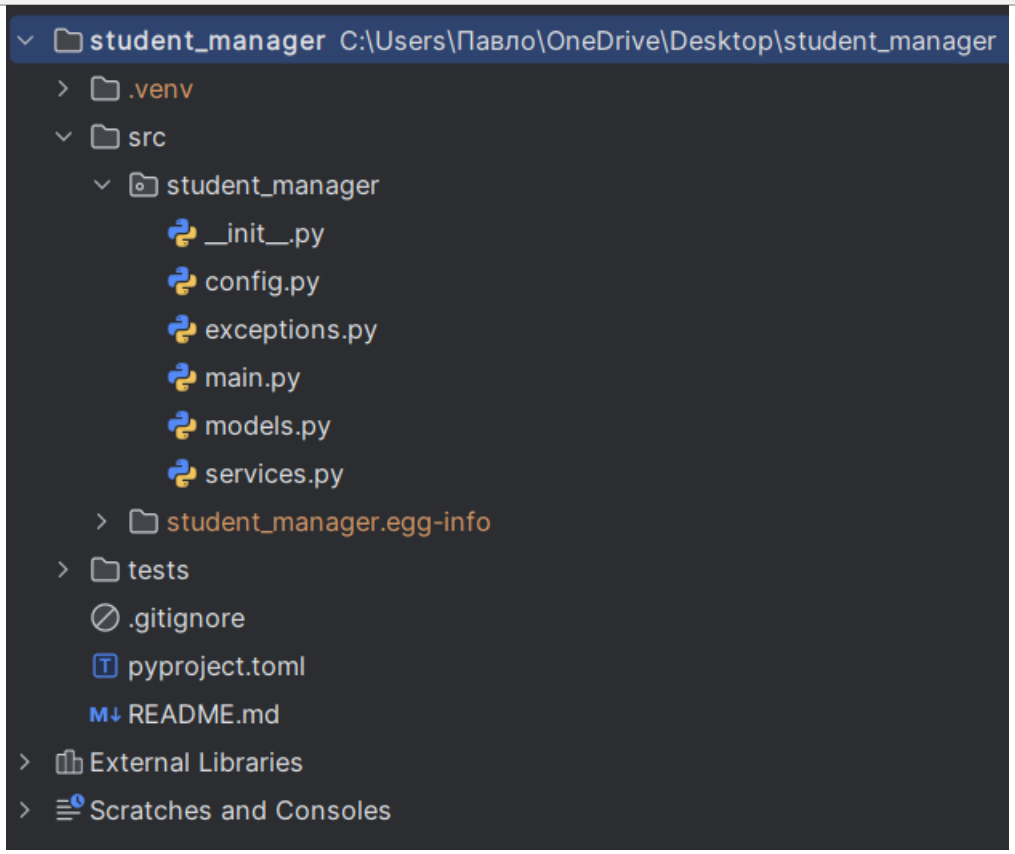


Рис. 1. Структура проєкту в PyCharm

Перевага src-layout полягає в тому, що пакет не імпортується випадково з поточного каталогу: для запуску його потрібно встановити (наприклад, у режимі editable), тому розробник тестує саме той пакет, який буде встановлено користувачам.

4. Virtual environment та встановлення проєкту

Для ізоляції залежностей створено virtual environment, його активовано, а проєкт встановлено в режимі editable:

```
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate        # Linux/macOS
python --version
python -m pip install -e .
python -m pip list
```

```
(.venv) PS C:\Users\Павло\OneDrive\Desktop\student_manager> python -m pip list
Package         Version Editable project location
--------------- ------- ------------------------------------------
pip             24.3.1
student-manager 0.1.0   C:\Users\Павло\OneDrive\Desktop\student_manager
```

Рис. 2. Пакет student-manager встановлено в editable-режимі

Каталог .venv не додається до Git-репозиторію, оскільки його вміст залежить від конкретного комп'ютера і може бути відтворений командою python -m venv .venv.

5. Опис створених модулів

5.1. Файл __init__.py

Модуль позначає каталог student_manager як Python package і містить версію пакета.

```
"""Student manager package."""

__version__ = "0.1.0"
```

5.2. Файл config.py

Модуль містить конфігураційні константи: допустимий діапазон середнього бала (MIN_GRADE, MAX_GRADE) та групу за замовчуванням (DEFAULT_GROUP).

```
"""Configuration constants of the application."""

MIN_GRADE: float = 0.0
MAX_GRADE: float = 100.0
DEFAULT_GROUP: str = "PI-21"
```

5.3. Файл exceptions.py

Модуль визначає власний виняток InvalidStudentDataError для некоректних даних студента.

```
"""Custom exceptions of the application."""



class InvalidStudentDataError(ValueError):
    """Raised when student data does not pass validation."""
```

5.4. Файл models.py

Модуль містить data model Student (dataclass) із валідацією в __post_init__ та властивістю full_name.

```
"""Data models."""

from dataclasses import dataclass

from student_manager.config import MAX_GRADE, MIN_GRADE
from student_manager.exceptions import InvalidStudentDataError



@dataclass
class Student:
    """Student with basic academic information."""

    first_name: str
    last_name: str
    group: str
    average_grade: float

    def __post_init__(self) -> None:
        if not self.first_name.strip() or not self.last_name.strip():
            raise InvalidStudentDataError("Name and surname must not be empty.")
        if not self.group.strip():
            raise InvalidStudentDataError("Group must not be empty.")
        if not MIN_GRADE <= self.average_grade <= MAX_GRADE:
            raise InvalidStudentDataError(
                f"Average grade must be in range {MIN_GRADE}..{MAX_GRADE}."
            )

    @property
    def full_name(self) -> str:
        return f"{self.first_name} {self.last_name}"
```

5.5. Файл services.py

Модуль містить бізнес-логіку: додавання, пошук, фільтрацію за групою, пошук за кількома критеріями, сортування, обчислення середнього бала групи та пошук найкращого студента.

```
"""Business logic: operations over the list of students."""
```

```
from student_manager.models import Student



def add_student(students: list[Student], student: Student) -> None:
    """Add a student to the list."""
    students.append(student)



def calculate_group_average(students: list[Student]) -> float:
    """Return the average grade of the given students."""
    if not students:
        return 0.0

    total = sum(student.average_grade for student in students)

    return total / len(students)



def find_best_student(students: list[Student]) -> Student | None:
    """Return the student with the highest average grade."""
    if not students:
        return None

    return max(students, key=lambda student: student.average_grade)



def filter_by_group(students: list[Student], group: str) -> list[Student]:
    """Return students of the given group."""
    return [student for student in students if student.group == group]



def find_student(students: list[Student], last_name: str) -> Student | None:
    """Find a student by last name (case-insensitive)."""
    for student in students:
        if student.last_name.lower() == last_name.lower():
            return student

    return None



def search_students(
    students: list[Student],
    group: str | None = None,
    min_grade: float | None = None,
) -> list[Student]:
    """Search by several criteria: group and/or minimal average grade."""
    result = students

    if group is not None:
        result = filter_by_group(result, group)

    if min_grade is not None:
        result = [s for s in result if s.average_grade >= min_grade]

    return result
```

```
def sort_by_grade(
    students: list[Student], descending: bool = True
) -> list[Student]:
    """Return a new list sorted by average grade."""
    return sorted(
        students,
        key=lambda student: student.average_grade,
        reverse=descending,
    )
```

5.6. Файл main.py

Модуль є точкою входу: формує демонстраційні дані, виводить результати та запускає меню; запуск захищено конструкцією if __name__ == "__main__".

```
"""Application entry point."""

import sys

from student_manager.config import DEFAULT_GROUP
from student_manager.exceptions import InvalidStudentDataError
from student_manager.models import Student
from student_manager.services import (
    add_student,
    calculate_group_average,
    filter_by_group,
    find_best_student,
    find_student,
    sort_by_grade,
)



def create_demo_students() -> list[Student]:
    """Create demo data."""
    return [
        Student("Anna", "Koval", "PI-21", 91.5),
        Student("Oleh", "Melnyk", "PI-21", 84.0),
        Student("Iryna", "Bondar", "PI-22", 95.2),
        Student("Andrii", "Shevchenko", "PI-21", 88.7),
    ]



def print_students(students: list[Student]) -> None:
    """Print students as a table."""
    print("\nStudents:")

    for student in students:
        print(
            f"{student.full_name:25}"
            f"{student.group:10}"
            f"{student.average_grade:6.2f}"
        )



def run_demo() -> None:
    """Non-interactive demonstration of all main operations."""
```

```
    students = create_demo_students()

    print_students(students)

    group = DEFAULT_GROUP
    group_students = filter_by_group(students, group)

    print(f"\nStudents of group {group}:")
    print_students(group_students)

    average = calculate_group_average(group_students)
    print(f"\nGroup average: {average:.2f}")

    best = find_best_student(students)
    if best is not None:
        print("\nBest student:", best.full_name, f"({best.average_grade:.2f})")

    student = find_student(students, "Melnyk")
    if student is not None:
        print("\nStudent found:", student.full_name, student.group)

    print("\nSorted by average grade (descending):")
    print_students(sort_by_grade(students))



def print_menu() -> None:
    """Print the menu."""
    print()
    print("1. Show all students")
    print("2. Find student")
    print("3. Show group")
    print("4. Show best student")
    print("5. Add student")
    print("6. Exit")



def read_new_student() -> Student:
    """Read a student from the keyboard with validation."""
    first_name = input("First name: ").strip()
    last_name = input("Last name: ").strip()
    group = input("Group: ").strip()
    grade_text = input("Average grade: ").strip().replace(",", ".")

    try:
        grade = float(grade_text)
    except ValueError as error:
        raise InvalidStudentDataError("Average grade must be a number.") from
error

    return Student(first_name, last_name, group, grade)



def run_menu(students: list[Student]) -> None:
    """Menu-driven interface."""
    while True:
        print_menu()
        command = input("Select command: ").strip()

        if command == "1":
```

```
            print_students(students)

        elif command == "2":
            student = find_student(students, input("Last name: "))
            if student:
                print(student.full_name, student.group, student.average_grade)
            else:
                print("Student not found.")

        elif command == "3":
            print_students(filter_by_group(students, input("Group: ")))

        elif command == "4":
            best = find_best_student(students)
            if best:
                print("Best:", best.full_name, best.average_grade)

        elif command == "5":
            try:
                add_student(students, read_new_student())
                print("Student added.")
            except InvalidStudentDataError as error:
                print(f"Error: {error}")

        elif command == "6":
            print("Goodbye.")
            break

        else:
            print("Unknown command.")



def main() -> None:
    """Run demo by default or the menu with --menu."""
    if "--menu" in sys.argv[1:]:
        run_menu(create_demo_students())
    else:
        run_demo()



if __name__ == "__main__":
    main()
```

6. Файл pyproject.toml

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"


[project]
name = "student-manager"
version = "0.1.0"
description = "Student management laboratory project"
requires-python = ">=3.11"


[project.scripts]
student-manager = "student_manager.main:main"
```

```
[tool.setuptools.packages.find]
where = ["src"]
```

Секція [build-system] визначає інструмент збирання пакета (setuptools). Секція [project] містить метадані проєкту: назву, версію, опис та мінімальну версію Python (3.11). Секція [project.scripts] створює консольну команду student-manager, яка викликає функцію main із модуля student_manager.main. Секція [tool.setuptools.packages.find] вказує, що пакети потрібно шукати в каталозі src.

7. Файл .gitignore

```
.venv/
__pycache__/
*.pyc
.pytest_cache/
.idea/
.vscode/
dist/
build/
*.egg-info/
.env
```

Файл виключає з репозиторію virtual environment, кеші Python та pytest, конфігурацію IDE, артефакти збирання та файл .env, який може містити секрети.

8. Фрагмент README.md

```
# Student Manager

Laboratory project for the course
"Professional Python".

## Description

Console application for managing students
and calculating academic statistics.

## Requirements

Python 3.11+

## Installation

    python -m venv .venv

Activate the virtual environment
(Windows: `.venv\Scripts\activate`, Linux/macOS: `source .venv/bin/activate`).
```

```
    python -m pip install -e .

## Run

    python -m student_manager.main

Interactive menu:

    python -m student_manager.main --menu
```

9. Запуск застосунку

Застосунок запускається як Python module командою python -m student_manager.main або через консольну команду student-manager.

```
(.venv) PS C:\Users\Павло\OneDrive\Desktop\student_manager> python -m student_manager.main

Students:
Anna Koval              PI-21      91.50
Oleh Melnyk             PI-21      84.00
Iryna Bondar            PI-22      95.20
Andrii Shevchenko       PI-21      88.70

Students of group PI-21:

Students:
Anna Koval              PI-21      91.50
Oleh Melnyk             PI-21      84.00
Andrii Shevchenko       PI-21      88.70

Group average: 88.07

Best student: Iryna Bondar (95.20)

Student found: Oleh Melnyk PI-21

Sorted by average grade (descending):

Students:
Iryna Bondar            PI-22      95.20
Anna Koval              PI-21      91.50
Andrii Shevchenko       PI-21      88.70
Oleh Melnyk             PI-21      84.00
```

Рис. 3. Результат запуску python -m student_manager.main

У режимі --menu доступне інтерактивне меню. На знімку нижче показано пошук студента за прізвищем та відхилення некоректного середнього бала (150) завдяки валідації.

```
(.venv) PS C:\Users\Павло\OneDrive\Desktop\student_manager> python -m student_manager.main --menu

1. Show all students
2. Find student
3. Show group
4. Show best student
5. Add student
6. Exit
Select command: 2
Last name: Melnyk
Oleh Melnyk PI-21 84.0

1. Show all students
2. Find student
3. Show group
4. Show best student
5. Add student
6. Exit
Select command: 5
First name: Test
Last name: User
Group: PI-21
Average grade: 150
Error: Average grade must be in range 0.0..100.0.

1. Show all students
2. Find student
3. Show group
4. Show best student
5. Add student
6. Exit
Select command: 6
Goodbye.
```

Рис. 4. Робота меню: пошук студента та валідація введення

9.1. Автоматизовані тести

```
(.venv) PS C:\Users\Павло\OneDrive\Desktop\student_manager> python -m unittest discover -s tests -v
test_average (test_services.ServicesTest.test_average) ... ok
test_best (test_services.ServicesTest.test_best) ... ok
test_find_and_search (test_services.ServicesTest.test_find_and_search) ... ok
test_sort (test_services.ServicesTest.test_sort) ... ok
test_validation (test_services.ServicesTest.test_validation) ... ok

----------------------------------------------------------------------
Ran 5 tests in 0.001s

OK
```

Рис. 5. Результат запуску unit-тестів

10. Робота з Git

Репозиторій ініціалізовано командою git init. Роботу розбито на логічні етапи, кожен із яких зафіксовано окремим commit:

```
git init
git add .gitignore pyproject.toml src/student_manager/__init__.py
git commit -m "Create project structure"


git add src/student_manager/models.py src/student_manager/services.py \
        src/student_manager/config.py src/student_manager/exceptions.py
git commit -m "Add student model and business logic"


git add src/student_manager/main.py README.md
git commit -m "Add application entry point and documentation"


git add tests/test_services.py
git commit -m "Add unit tests"
```

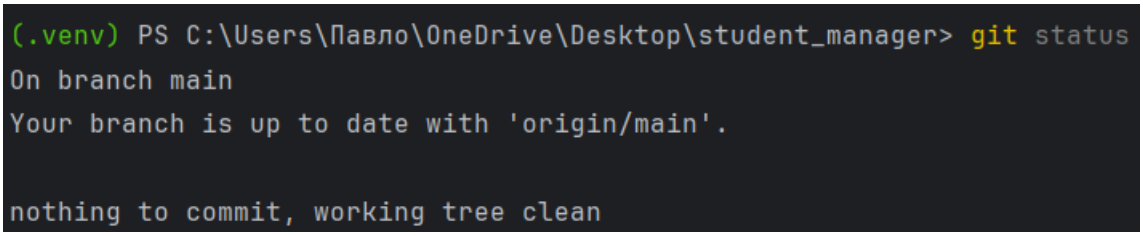


Рис. 6. Результат git status

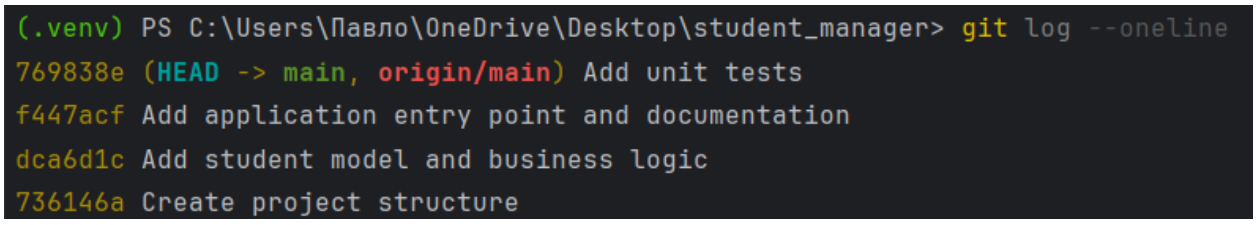


Рис. 7. Результат git log --oneline

Репозиторій опубліковано на GitHub: https://github.com/Pavlo12112/professional-python-lab1

Для публікації виконано команди:

```
git branch -M main
git remote add origin https://github.com/Pavlo12112/professional-python-lab1.git
git push -u origin main
```

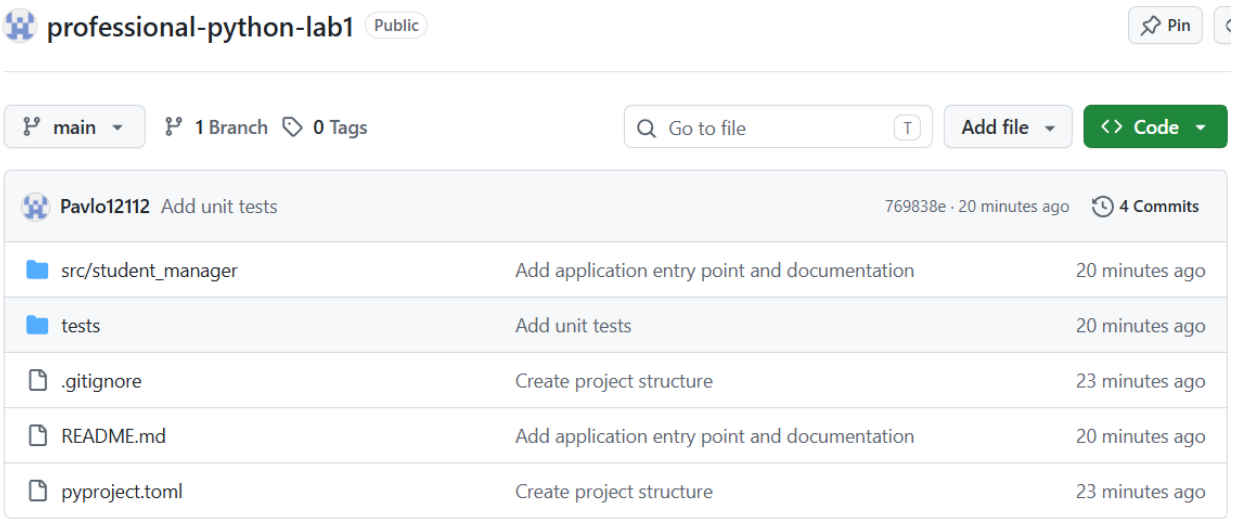


Рис. 8. Репозиторій на GitHub

11. Відповіді на контрольні запитання

1. Що таке Python-проєкт?

Це структурований набір файлів і каталогів (код, тести, документація, конфігурація, залежності), організований так, щоб його можна було розробляти, тестувати, встановлювати та супроводжувати.

2. Чим проєкт відрізняється від одного Python script?

Скрипт — це один файл для вирішення невеликої задачі. Проєкт розділений на модулі й пакети, має тести, конфігурацію (pyproject.toml), документацію, контроль версій та керовані залежності.

3. Що таке Python module?

Це звичайний файл із розширенням .py, що містить функції, класи та змінні й може бути імпортований в інші файли.

4. Що таке Python package?

Це каталог із Python-модулями (зазвичай із файлом __init__.py), який об’єднує споріднені модулі під спільним ім’ям.

5. Для чого потрібний __init__.py?

Він позначає каталог як пакет, виконується під час імпорту пакета і може містити ініціалізаційний код, версію або визначати публічний інтерфейс пакета.

6. Що таке src-layout?

Це структура проєкту, у якій пакет розміщується не в корені репозиторію, а в підкаталозі src/<назва_пакета>.

7. Які переваги має src-layout?

Запобігає випадковому імпорту пакета з поточного каталогу, змушує тестувати встановлений пакет, чітко відокремлює код від тестів і службових файлів, зменшує конфлікти імен.

8. Для чого використовується virtual environment?

Для ізоляції бібліотек і версій Python одного проєкту від інших проєктів та від системного Python, щоб уникати конфліктів залежностей.

9. Як створити .venv?

Командою python -m venv .venv у кореневому каталозі проєкту.

10. Як активувати virtual environment?

У Windows — .venv\Scripts\activate, у Linux/macOS — source .venv/bin/activate.

11. Для чого використовується pip?

pip — менеджер пакетів Python: він встановлює, оновлює та видаляє бібліотеки з Python Package Index та інших джерел.

12. Чому рекомендується python -m pip?

Така команда гарантовано запускає pip саме для того інтерпретатора Python, який активний (наприклад, у віртуальному середовищі), і не залежить від того, який pip знайдено в PATH.

13. Для чого потрібний pyproject.toml?

Це стандартний файл метаданих і конфігурації проєкту: у ньому описують систему збирання, назву, версію, залежності, консольні команди та налаштування інструментів.

14. Що містить секція [project]?

Метадані проєкту: name, version, description, requires-python, залежності (dependencies), автори, ліцензію та інші поля.

15. Що таке editable installation?

Встановлення пакета командою pip install -e ., за якого в середовищі створюється посилання на вихідний код. Зміни у файлах діють одразу без перевстановлення.

16. Для чого використовується if __name__ == "__main__"?

Щоб код запуску виконувався лише тоді, коли файл запущено безпосередньо, і не виконувався під час імпорту модуля в інший файл.

17. Що таке Git?

Розподілена система контролю версій, яка зберігає історію змін, дозволяє повертатися до попередніх версій, працювати в команді та використовувати гілки.

18. Що таке repository?

Сховище проєкту, що містить файли та повну історію їх змін (локальний каталог .git або віддалений сервіс на кшталт GitHub чи GitLab).

19. Що таке commit?

Зафіксований знімок стану проєкту з унікальним ідентифікатором, автором, датою та повідомленням, що описує зміни.

20. Для чого використовується git status?

Показує поточний стан робочого каталогу: які файли змінено, які додано до індексу, які не відстежуються.

21. Для чого git add?

Додає зміни вказаних файлів до індексу (staging area), тобто готує їх до включення в наступний commit.

22. Для чого git commit?

Створює commit із проіндексованими змінами та повідомленням, зберігаючи їх в історії репозиторію.

23. Що показує git log?

Історію commits: ідентифікатори, авторів, дати та повідомлення; з параметром --oneline — компактний список по одному рядку на commit.

24. Для чого потрібен .gitignore?

Визначає файли й каталоги, які Git не повинен відстежувати: virtual environment, кеші, налаштування IDE, артефакти збирання, секрети.

25. Чому .venv не потрібно комітити?

Він великий, залежить від ОС і шляхів конкретного комп’ютера та відтворюється з опису залежностей. Його додавання засмічує репозиторій і ламає переносимість.

26. Для чого потрібний README?

Це вхідна документація проєкту: опис, вимоги, інструкція зі встановлення й запуску, структура проєкту та інформація про автора.

27. Що таке type hints?

Анотації типів параметрів і значень, що повертаються (наприклад, list[Student], float | None). Вони не впливають на виконання, але покращують читабельність і дозволяють статичним аналізаторам знаходити помилки.

28. Для чого програму розділяють на modules?

Для зменшення складності, повторного використання коду, зручнішого тестування, командної роботи та чіткого розподілу відповідальності.

29. Що таке business logic?

Частина програми, що реалізує правила й обчислення предметної області (у цій роботі — пошук, фільтрація, сортування та підрахунок статистики студентів), незалежно від введення-виведення.

30. Чому потрібно відокремлювати application entry point від business logic?

Бізнес-логіку можна тестувати й повторно використовувати без запуску застосунку та без залежності від консолі чи інтерфейсу. Точка входу відповідає лише за запуск і взаємодію з користувачем.

12. Висновок

У лабораторній роботі створено консольний застосунок «Система обліку студентів» як повноцінний Python-проєкт. Застосовано src-layout, розділення коду на модулі (models, services, config, exceptions, main), data model на основі dataclass, type hints, власний exception та валідацію даних. Налаштовано virtual environment, файл pyproject.toml із консольною командою, встановлення в editable-режимі, а також файли .gitignore і README.md.

Розробку зафіксовано в Git-репозиторії у вигляді чотирьох логічних commits, репозиторій опубліковано на GitHub. Працездатність перевірено запуском застосунку як Python module та через консольну команду, а також автоматизованими тестами. Отриману структуру проєкту доцільно використовувати як основу для наступних лабораторних робіт курсу.